

과제 보고서

게임프로그래밍기초

Dead City

건물 내부 좀비 전투 시스템

이름	강우현
학번	202404178
학과	인공지능융합공학부

차 례

I. 서론

- 프로젝트 배경 및 목적
- 초기 프로젝트 개요 (FPS_AIM)
- 최종 프로젝트 전환 배경

II. 본론

- 수업 연계 핵심 개념
- 시스템 구조 설계
- 핵심 스크립트 상세 분석
- UI 시스템 구현

5. 주요 이슈 및 해결 과정

III. 결론

I. 서론

1. 프로젝트 배경 및 목적

본 프로젝트는 게임프로그래밍기초 수업의 기말 과제로, Unity 엔진을 활용하여 3D 게임을 제작하는 것을 목표로 한다. 수업 전반에 걸쳐 학습한 상속, 인터페이스, 싱글턴, 코루틴, 레이캐스트, NavMesh AI, 이벤트 시스템 등의 개념을 실제 게임 구현에 적용하고, 이를 통해 게임 프로그래밍의 핵심 패턴을 체득하는 것을 목적으로 한다.

2. 초기 프로젝트 개요 (FPS_AIM)

기말 과제의 초기 방향은 1인칭 슈팅 에임 트레이닝 게임(FPS_AIM)으로 설정하였다. 해당 프로젝트의 주요 구성은 다음과 같다.

항목	내용
과제명	Unity FPS 에임 연습 게임
모드 구성	Easy(고정 타겟 3개) / Normal(이동 타겟 2개) / Hard(랜덤 위치 소형 타겟 3개)
타겟 수 선택	10 / 20 / 30개 선택 가능
핵심 스크립트	GameManager, SelectionUI, InGameUI, ResultUI, PauseUI, MouseLook, Shooting, Bullet, Target

초기 프로젝트는 게임 UI 흐름(SelectionPanel → InGamePanel → ResultPanel), 싱글턴 기반 GameManager, 1인칭 카메라 제어(MouseLook), 총알 물리(Bullet), 타겟 이동(코루틴 활용) 등을 구현하였다. 그러나 구현 과정에서 수업 시간에 배운 핵심 개념들, 즉 상속 구조, NavMesh AI, IDamageable 인터페이스, LivingEntity 기반 클래스 등이 활용되지 않아 수업 내용과의 연계성이 부족하다는 점을 인식하게 되었다.

3. 최종 프로젝트 전환 배경

이러한 문제를 해결하기 위해 프로젝트의 방향을 수업에서 직접 다룬 좀비 서바이벌 게임 구조를 기반으로 한 새로운 프로젝트로 전환하였다. 영화 '반도'의 세계관을 참고하여, 생존자들이 도시 건물 내부를 탐험하며 층별로 증가하는 좀비를 처치하는 3인칭 슈팅 게임 Dead City: Island Z의 핵심 씬인 건물 내부 전투 시스템을 구현하였다.

최종 프로젝트는 수업 3주차부터 17주차까지 다룬 클래스 설계, 물리 엔진, 애니메이션, 레이캐스트, 인터페이스, 상속, 내비게이션 AI, 싱글턴, 이벤트 시스템을 전반적으로 활용하여 수업 학습 내용과의 연계성을 크게 높였다.

II. 본론

1. 수업 연계 핵심 개념

최종 프로젝트에서 활용한 수업 개념과 구현 위치를 정리하면 다음과 같다.

수업 주차	개념	프로젝트 적용 내용
3주차	클래스 / 컴포넌트	GetComponent로 Rigidbody, Animator, NavMeshAgent 등 컴포넌트 참조
7주차	이동 / 충돌	PlayerMovement: Rigidbody.MovePosition으로 WASD 이동 구현
8주차	OnTrigger / 프리팹	ElevatorInteraction의 OnTriggerEnter/Exit, Instantiate로 좀비 스폰
11주차	애니메이션	Animator Parameter(Speed, Attack, Die, Hit)와 Transition 조건 설정
15주차	레이캐스트 / 인터페이스 / 코루틴	Gun의 발사 레이캐스트, IDamageable 인터페이스로 느슨한 커플링, SpawnZombies 코루틴
16주차	상속 / 다형성 / NavMesh	LivingEntity 기반 PlayerHealth·Zombie 상속, override 피격·사망, NavMeshAgent 추적
17주차	싱글톤 / 이벤트	UIManager·GameManager·FloorManager 싱글톤, zombie.onDeath 이벤트로 카운트 차감

2. 시스템 구조 설계

프로젝트의 전체 스크립트 구조는 다음과 같이 설계하였다.

분류	스크립트	역할
생명체 기반	LivingEntity.cs	체력 관리, onDeath 이벤트, IDamageable 인터페이스 구현
플레이어	PlayerMovement.cs	카메라 기준 WASD 이동, 마우스 방향 회전(레이캐스트)
플레이어	PlayerHealth.cs	LivingEntity 상속, 체력 UI 연동, 사망 시 GameManager 호출

플레이어	PlayerShooter.cs	총 발사·재장전·무기 교체(1/2/3번), IK 손 위치 보정
좀비 AI	Zombie.cs	LivingEntity 상속, NavMesh 추적, 근접 공격, 머리 위 체력바
좀비 AI	ZombieData.cs	ScriptableObject로 층별 좀비 능력치 데이터 분리 관리
층 관리	FloorManager.cs	5층 구성, 코루틴 순차 스폰, onDeath 이벤트로 카운트 관리
상호작용	ElevatorInteraction.cs	IInteractable 인터페이스, OnTrigger F키 입력으로 다음 층 이동
UI 관리	UIManager.cs	싱글턴, 체력·배고픔·햇바·층 정보·좀비 수 UI 통합 관리
게임 상태	GameManager.cs	싱글턴, 게임오버·클리어 상태 관리, 씬 재시작

3. 핵심 스크립트 상세 분석

1) LivingEntity — 생명체 기반 클래스 (16주차 상속·다형성)

플레이어와 좀비가 공통으로 상속하는 기반 클래스이다. IDamageable 인터페이스를 구현하여 총이 맞힌 대상의 타입에 관계없이 데미지를 전달할 수 있도록 느슨한 커플링을 적용하였다.

```
public class LivingEntity : MonoBehaviour, IDamageable
{
    public float health { get; protected set; }
    public bool dead { get; protected set; }
    public event Action onDeath; // 이벤트 (17주차)
    public virtual void OnDamage(float damage, ...) { ... } // 가상 메서드
    public virtual void Die() { onDeath?.Invoke(); dead = true; }
}
```

2) Zombie — NavMesh AI + LivingEntity 상속 (16주차)

LivingEntity를 상속받아 OnDamage와 Die를 override로 재정의하고, NavMeshAgent를 사용하여 플레이어를 자동으로 추적한다. 경로 갱신은 코루틴을 통해 0.25초 주기로 수행하여 성능을 최적화하였으며, 머리 위 체력바는 Canvas를 World Space로 설정하고 매 프레임 카메라 방향을 바라보도록 구현하였다.

```
private IEnumerator UpdatePath() // 코루틴 - 15주차
{
    while (!dead)
    {
```

```

        navMeshAgent.SetDestination(targetEntity.transform.position);
        yield return new WaitForSeconds(0.25f);
    }
}

```

3) FloorManager — 5층 구성 및 코루틴 스폰 (15주차·17주차)

총 5층으로 구성되며 층마다 좀비 수가 2마리씩 증가한다(1층: 2마리 ~ 5층: 10마리). 코루틴을 활용하여 좀비를 순차적으로 스폰하고, zombie.onDeath 이벤트 구독을 통해 좀비 사망 시 카운트를 자동으로 차감한다. 층 클리어 시 F키 입력 대기 상태로 전환되며, 5층 클리어 시 게임 클리어를 호출한다.

층	좀비 수	능력치 배율
1층	2마리	기본값
2층	4마리	체력 +30
3층	6마리	체력 +60
4층	8마리	체력 +90
5층	10마리	체력 +120

4) PlayerMovement — 카메라 기준 이동 + 레이캐스트 회전 (7주차·15주차)

WASD 이동 시 카메라의 forward/right 벡터에서 Y축 성분을 제거하고 정규화하여, 카메라가 기울어진 환경에서도 화면 기준으로 정확하게 이동하도록 구현하였다. 마우스 방향 회전은 Ground 레이어로 레이캐스트를 쏘아 커서가 향하는 바닥 위치를 계산한 후 Quaternion.Slerp로 부드럽게 회전한다.

```

Vector3 camForward = mainCamera.transform.forward;
camForward.y = 0f; camForward.Normalize();
Vector3 moveDir = (camForward * move + camRight * rotate).normalized;

```

5) PlayerShooter — 무기 교체 시스템 (15주차·11주차)

1.2.3번 키로 권총·라이플·샷건을 교체할 수 있다. EquipWeapon() 메서드에서 GunData(능력치) 교체와 무기 모델 활성화를 동시에 처리하며, 무기 모델 내부의 LeftHandMount·RightHandMount Transform을 자동으로 찾아 OnAnimatorIK의 IK 타겟을 갱신한다.

4. UI 시스템 구현

UIManager는 싱글턴 패턴으로 구현되어 모든 스크립트에서 UIManager.instance를 통해 접근할 수 있다. UI 구성은 다음과 같다.

위치	구성 요소	내용
좌상단	체력 / 배고픔 바	Slider + TextMeshPro로 HP, Food 실시간 표시
우상단	층 / 좀비 수 / 탄약	현재 층(N/5층), 남은 좀비 수, 탄창/잔탄 표시
하단 중앙	햇바 (1·2·3번)	선택된 슬롯 노란색 강조, 무기명 표시
화면 중앙	상호작용 힌트	층 클리어 후 "F를 눌러 다음 층으로 이동하세요" 표시
화면 중앙	층 클리어 메시지	"N층 클리어!" 2.5초 표시 후 자동 숨김
전체 오버레이	게임오버 / 클리어	반투명 패널 + 재시작 버튼 (GameManager.RestartGame)

5. 주요 이슈 및 해결 과정

이슈	해결 방법
캐릭터 누워서 생성	Meshy AI FBX는 Z-up 좌표계 → Bake Axis Conversion 체크, 모델에 빈 부모로 감싸고 Rotation X: 90 적용
NavMesh Agent 에러	바닥 Static → Navigation Static 설정 후 Bake. Rigidbody Is Kinematic 체크로 NavMesh와 충돌 방지
Input System 에러	Player Settings → Active Input Handling → Both 설정으로 구버전·신버전 입력 동시 지원
클래스 중복 정의 에러	Assets 전체에서 동명 스크립트 검색, Scripts 폴더 외 중복 파일 삭제
좀비 체력바 미추적	HealthBarCanvas를 World Space Canvas로 설정, Update()에서 mainCamera.transform.rotation 할당
탄약 UI 미갱신	UIManager에 UpdateAmmoText(magAmmo, ammoRemain) 메서드 추가, PlayerShooter Update()에서 매 프레임 호출
좌우 이동 시 애니메이션 미출력	Animator Speed 파라미터를 move(W/S)만 사용하던 것을 Vector2(rotate, move).magnitude로 변경

III. 결론

본 프로젝트는 게임프로그래밍기초 수업에서 학습한 핵심 개념들을 Unity 3D 게임 개발에 직접 적용하는 것을 목표로 하였다. 초기 프로젝트(FPS_AIM)는 1인칭 에임 트레이닝 게임으로서 완성도 면에서는 일정 수준에 도달하였으나, 수업에서 다룬 상속 구조, NavMesh AI, 인터페이스 등 핵심 개념이 거의 활용되지 않았다는 한계가 있었다.

이를 보완하기 위해 최종 프로젝트 Dead City: Island Z의 건물 내부 전투 씬으로 방향을 전환하였으며, 3주차부터 17주차까지의 수업 내용을 체계적으로 반영하였다. 구체적으로는 LivingEntity 상속 구조, IDamageable 인터페이스, NavMeshAgent 기반 좀비 AI, 코루틴을 활용한 순차 스폰, onDeath 이벤트로 층 클리어 감지, UIManager·GameManager·FloorManager의 싱글턴 패턴을 구현하였다.

제작 과정에서 Meshy AI 모델의 축 방향 문제, NavMesh 설정, Input System 충돌, 스크립트 중복 오류 등 다양한 실제 개발 이슈를 경험하고 해결함으로써 Unity 게임 개발의 전반적인 워크플로우를 이해할 수 있었다.

향후 과제로는 각 층마다 다른 맵 환경 적용, 보스 좀비 추가, 아이템(체력팩, 탄약) 드롭 시스템, 메인 메뉴 씬과의 멀티씬 연동 등을 통해 완성도를 높일 수 있을 것으로 기대한다.

※ 본 보고서에 수록된 모든 코드는 수업 강의자료를 참고하여 직접 작성하였음.